SAE 401

Conception et modélisation d'un outil automatisé d'audit de cybersécurité

Dorian Maissonnier
Jérémy Guibert
Luderic BROSSE
Mattéo Recoquillon
Thibault Rommes
Louann Petit

NICE CÔTE D'AZUR

SOMMAIRE

Présentation du projet **3**
I - Les choix techniques **4**
I.1 - Choix des outils de scan 4
I.2 - Système de scoring 11
I.3 - Conception de l'architecture de la plateforme SaaS 14
II - Création de l'orchestrateur **15**
II.1 - Les contraintes technique 16
II.2 - La standardisation des données (La fusion des résultats) 18
II.3 - L'intégration du Scoring et la génération du livrable PDF 19
Conclusion **23**

Présentation du projet

Ce projet vise à rendre l'audit de cybersécurité aussi lisible et accessible qu'un rapport SEO pour un client non expert . Pour y parvenir, nous avons conçu une plateforme SaaS d'audit automatisé en « boîte noire ». Son développement s'est articulé autour de trois étapes clés, d'abord la sélection justifiée d'outils de scan open-source, ensuite la création d'un système de scoring vulgarisé, basé sur la matrice de l'ANSSI croisant l'impact et la difficulté d'exploitation des failles, et enfin la modélisation d'un orchestrateur central. Ce dernier pilote les scans selon deux modes (simple ou avancé) , tout en imposant une approche éthique « White Hat » avec temporisation pour éviter tout déni de service involontaire sur l'infrastructure cible.

I - Les choix techniques

I.1 - Choix des outils de scan

Tableau des outils trié PAR POIDS : pré-scan / scan :

Rapidité	Outils Pré-scan	Poids
5) 5 à 15 secondes	Google Dorks	0 requêtes
4) 2 à 5 secondes	CTFR	0 requêtes
8) 15 secondes à 1 minute	Email Harvester	0 requêtes
2) Moins d'une seconde	Whois	1 à 3 requêtes réseau (qqs octets)
1) Quelques millisecondes	Dig	1 à 3 requêtes réseau (qqs octets)
3) 1 à 3 secondes	WhatWeb	1 à 5 requêtes HTTP
9) Quelques minutes	Pymeta	qqs Mo de téléchargement
10) Plusieurs minutes	Knockpy	milliers de requêtes DNS/secondes
	Outils Scan	**Poids**
7) 10 secondes à 1 minute	GitLeaks	1 à 10 Mo
6) 10 à 30 secondes	Lighthouse	50 à 150 requêtes HTTP (1 à 10 Mo de téléchargement)
11) 5 à 15 minutes	Nikto	+ de 6 700 requêtes HTTP spécifiques
12) 15 minutes à plusieurs heures	Dirsearch (Dirb)	10 000 à 100 000+ requêtes HTTP

Classement Rapidité

Léger
Moyen
Lourd

Pour offrir une bonne expérience utilisateur sur notre plateforme SaaS, nous avons classé les outils en deux catégories :

- Le pré-scan (Mode Basic) : Des outils furtifs et quasi instantanés.
- Le scan profond (Mode Avancé) : Des outils lourds, et chronophages.

Les outils de reconnaissance (Mode Basic) :

Nous avons regroupé les outils selon leur rapidité et leur discrétion :

- Les outils instantanés (Moins d'une seconde) :*Whois* et *Dig* ne nécessitent que 1 à 3 requêtes réseau pour récupérer des informations cruciales sur le domaine demandé.
- L'avantage : Ce mode permet au client d'avoir un premier état des lieux immédiat sans surcharger la bande passante ni alerter les pare-feux.

Whois :

Cet outil interroge les bases de données publiques pour trouver à qui appartient un nom de domaine, les contacts administratifs, et les serveurs DNS.

Test avec l'url de youtube :

```
The Registry database contains ONLY .COM, .NET, .EDU domains and
Registrars.
Domain Name: youtube.com
Registry Domain ID: 142504053_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2026-01-14T10:29:14+0000
Creation Date: 2005-02-15T05:13:12+0000
Registrar Registration Expiration Date: 2027-02-15T00:00:00+0000
Registrar: MarkMonitor, Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientUpdateProhibited (https://www.icann.org/epp#clientUpdateProhibited)
Domain Status: clientTransferProhibited (https://www.icann.org/epp#clientTransferProhibited)
Domain Status: clientDeleteProhibited (https://www.icann.org/epp#clientDeleteProhibited)
Domain Status: serverUpdateProhibited (https://www.icann.org/epp#serverUpdateProhibited)
Domain Status: serverTransferProhibited (https://www.icann.org/epp#serverTransferProhibited)
Domain Status: serverDeleteProhibited (https://www.icann.org/epp#serverDeleteProhibited)
Registrant Organization: Google LLC
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/youtube.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/youtube.com
Name Server: ns4.google.com
Name Server: ns2.google.com
Name Server: ns3.google.com
Name Server: ns1.google.com
DNSSEC: unsigned
URL of the ICANN WHOIS Data Problem Reporting System: http://wdprs.internic.net/
>>> Last update of WHOIS database: 2026-03-16T12:19:36+0000 <<<
```

Dig :

C’est un outil d'administration réseau. Il va "poser des questions" aux annuaires d'internet (les serveurs DNS) pour savoir où se trouvent physiquement les serveurs d'un site, ou quels sont ses serveurs d'e-mails.

Test avec l’url du site web de l’université :

```
root@test-outils:~# dig univ-cotedazur.fr

; <<>> DiG 9.20.18-1~deb13u1-Debian <<>> univ-cotedazur.fr
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62060
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;univ-cotedazur.fr.             IN      A

;; ANSWER SECTION:
univ-cotedazur.fr.      81265   IN      A       134.59.204.162

;; Query time: 7 msec
;; SERVER: 192.168.1.254#53(192.168.1.254) (UDP)
;; WHEN: Wed Mar 18 17:23:06 CET 2026
;; MSG SIZE  rcvd: 62

root@test-outils:~#
```

- Les outils très rapides (1 à 15 secondes) : *WhatWeb* (identification des technologies en 1 à 5 requêtes HTTP), *Google Dorks*, et *CTFR*.

WhatWeb :

C’est un outil de reconnaissance semi-passive qui identifie exactement les technologies qui font tourner un site web sans l'attaquer, permettant ainsi de mieux cibler les tests de sécurité.

Test sur un environnement intentionnellement vulnérable :

```
root@test-outils:~# whatweb http://localhost
http://localhost [302 Found] Apache[2.4.25], Cookies[PHPSESSID,security], HTTPServer[Debian Linux][Apache/2.4.25 (Debian)], IP[::1], Redirec
tLocation[login.php]
http://localhost/login.php [200 OK] Apache[2.4.25], Cookies[PHPSESSID,security], DVWA, HTTPServer[Debian Linux][Apache/2.4.25 (Debian)], IP[
::1], PasswordField[password], Title[Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*]
```

Google Dorks :

Ce n'est pas un logiciel, mais une technique de reconnaissance passive. Elle utilise des commandes de recherche avancées (comme site: ou filetype:) directement dans Google pour dénicher des informations sensibles exposées par erreur sur le web (mots de passe, sauvegardes, logs), le tout sans jamais interagir avec le serveur cible ni déclencher d'alerte.

CTFR :

C'est un outil brillant. Plutôt que d'attaquer le serveur, CTFR va interroger les registres publics mondiaux des certificats de sécurité (HTTPS). Chaque fois qu'une entreprise crée un sous-domaine sécurisé (ex: dev.youtube.com), elle doit le déclarer publiquement. CTFR lit ces journaux pour trouver les sous-domaines cachés.

Test en utilisant l'url de youtube :

```
root@test-outils:~/ctfr# python3 ctfr.py -d youtube.com
/root/ctfr/ctfr.py:29: SyntaxWarning: invalid escape sequence '\ '
  / ___|_   _|  ___|  _ \
/root/ctfr/ctfr.py:40: SyntaxWarning: invalid escape sequence '\.'
  return re.sub('.*www\.','',target,1).split('/')[0].strip()


          ____ _____ _____ ____
         / ___|_   _|  ___|  _ \
        | |     | | | |_  | |_) |
        | |___  | | |  _| |  _ <
         \____| |_| |_|   |_| \_\

     Version 1.2 - Hey don't miss AXFR!
    Made by Sheila A. Berta (UnaPibaGeek)

/root/ctfr/ctfr.py:40: DeprecationWarning: 'count' is passed as positional argument
  return re.sub('.*www\.','',target,1).split('/')[0].strip()

[!] ---- TARGET: youtube.com ---- [!]

[-]  *.c.youtube.com
[-]  *.cache1.c.youtube.com
*.cache2.c.youtube.com
*.cache3.c.youtube.com
*.cache4.c.youtube.com
*.cache5.c.youtube.com
*.cache6.c.youtube.com
*.cache7.c.youtube.com
*.cache8.c.youtube.com
*.c.youtube.com
[-]  *.cctldtest.youtube.com
cctldtest.youtube.com
[-]  *.gdata.youtube.com
uploads.stage.gdata.youtube.com
```

Les outils d'analyse profonde (Mode Avancé) :

Cette section est dédiée aux investigations en profondeur. Lorsque le client sélectionne l'option « Analyse mode avancé », l'orchestrateur déploie une série d'outils d'audit nettement plus exhaustifs et intrusifs :

- Les scanners de vulnérabilités et de contenus : *GitLeaks* (10s à 1 min, analyse de code), *Lighthouse* (10 à 30s, audit de performance et SEO), et *Pymeta* (quelques minutes, nécessitant quelques mégaoctets de téléchargement).
- L'avantage : Ce mode est indispensable pour déceler des failles critiques (Backend, Infrastructure) mais nécessite plus de ressources serveur et de temps d'attente.

GitLeaks :

Gitleaks est un outil d'analyse spécialisé qui sert à trouver des informations confidentielles codées en dur dans des lignes de code, tout en étant capable de localiser précisément la ligne vulnérable.

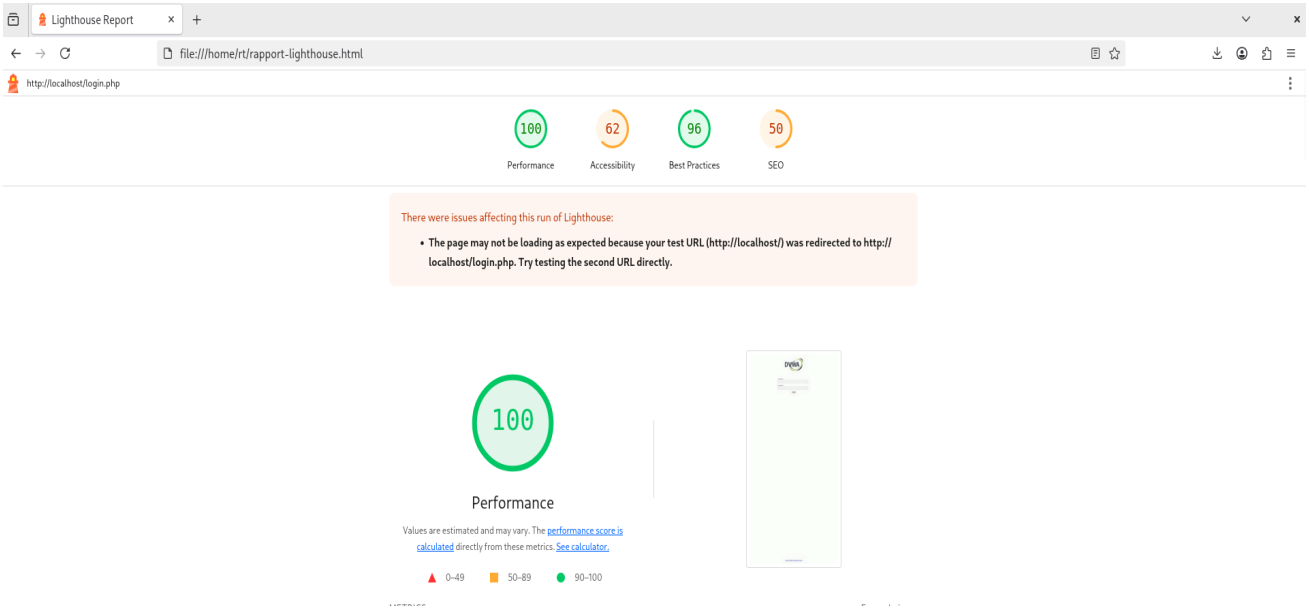
Résultat obtenu :

```
Finding:     ENV STRIPE_API_KEY="sk_live_1234567890abcdef12345678"
Secret:      sk_live_1234567890abcdef12345678"
RuleID:      stripe-access-token
Entropy:     4.438334
File:        Dockerfile
Line:        6
Fingerprint: Dockerfile:stripe-access-token:6
```

LightHouse :

C'est un outil qui audite une page web pour évaluer sa vitesse de chargement, son accessibilité et ses bonnes pratiques en référencement (SEO).

Résultat acquis :



Pymeta :

Il utilise des moteurs de recherche pour trouver et télécharger les documents publics d'un domaine cible (comme des PDF ou des fichiers Excel) afin d'en extraire les métadonnées et d'y débusquer des informations sensibles cachées, telles que des noms d'utilisateurs ou des versions de logiciels.

Test avec l'url du site web de l'université :

```
root@test-outils:/home/rt/pymeta# pymeta -d intranet-etu.univ-cotedazur.fr
/usr/local/lib/python3.13/dist-packages/pymetasec-1.2.0-py3.13.egg/pymeta/search.py:55: SyntaxWarning: invalid escape sequence '\)'
/usr/local/lib/python3.13/dist-packages/pymetasec-1.2.0-py3.13.egg/pymeta/search.py:55: SyntaxWarning: invalid escape sequence '\)'

PyMeta v1.2.0 - by @m8sec

[*] Target Domain     : intranet-etu.univ-cotedazur.fr
[*] Search Engines(s) : google, bing
[*] File Types(s)     : pdf, xls, xlsx, csv, doc, docx, ppt, pptx
[*] Max Downloads     : 50

[*] Searching google, bing for 8 file type(s) on "intranet-etu.univ-cotedazur.fr"
[*] 0   | pdf  - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:pdf&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | pdf  - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:pdf&first=0 (200)
[*] 0   | xls  - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:xls&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | xls  - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:xls&first=0 (200)
[*] 0   | xlsx - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:xlsx&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | xlsx - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:xlsx&first=0 (200)
[*] 0   | csv  - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:csv&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | csv  - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:csv&first=0 (200)
[*] 0   | doc  - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:doc&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | doc  - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:doc&first=0 (200)
[*] 0   | docx - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:docx&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | docx - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:docx&first=0 (200)
[*] 0   | ppt  - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:ppt&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | ppt  - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:ppt&first=0 (200)
[*] 0   | pptx - https://www.google.com/search?q=site:intranet-etu.univ-cotedazur.fr+filetype:pptx&num=100&start=0 (429)
[*] None 200 response, exiting search (429)
[*] 0   | pptx - http://www.bing.com/search?q=site:intranet-etu.univ-cotedazur.fr%20filetype:pptx&first=0 (200)
[*] No results found, closing...
root@test-outils:/home/rt/pymeta#
```

- Le Brute-Force : *Dirsearch (Dirb)*, qui est l'outil le plus lourd, pouvant prendre de 15 minutes à plusieurs heures en générant entre 10 000 et 100 000 requêtes HTTP.

Dirsearch :

C'est un outil de scan par force brute conçu pour découvrir les dossiers et les fichiers cachés d'un site web qui ne devraient normalement pas être accessibles au public.

Test effectué sur la loopback (127.0.0.1) :

```
root@test-outils:/home/rt# dirsearch -u http://127.0.0.1/
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

  _|. _ _  _  _  _ _|_    v0.4.3
 (_||| _) (/_(_|| (_| )

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460

Output File: /home/rt/reports/http_127.0.0.1/__26-03-19_15-53-39.txt

Target: http://127.0.0.1/

[15:53:39] Starting:
[15:53:43] 200 -   57B  - /.gitignore
[15:53:43] 403 -  295B  - /.ht_wsr.txt
[15:53:43] 403 -  298B  - /.htaccess.orig
[15:53:43] 403 -  298B  - /.htaccess.bak1
[15:53:43] 403 -  298B  - /.htaccess.save
[15:53:43] 403 -  300B  - /.htaccess.sample
[15:53:43] 403 -  299B  - /.htaccess_extra
[15:53:43] 403 -  296B  - /.htaccess_sc
[15:53:43] 403 -  298B  - /.htaccess_orig
[15:53:43] 403 -  296B  - /.htaccessBAK
[15:53:43] 403 -  296B  - /.htaccessOLD
[15:53:43] 403 -  297B  - /.htaccessOLD2
[15:53:43] 403 -  289B  - /.html
[15:53:43] 403 -  288B  - /.htm
[15:53:43] 403 -  298B  - /.htpasswd_test
[15:53:43] 403 -  294B  - /.htpasswds
[15:53:43] 403 -  295B  - /.httr-oauth
[15:53:45] 403 -  288B  - /.php
[15:53:45] 403 -  289B  - /.php3
[15:53:51] 200 -    2KB - /about.php
[15:54:10] 200 -    7KB - /CHANGELOG.md
[15:54:11] 301 -  307B  - /config  ->  http://127.0.0.1/config/
[15:54:12] 200 -  468B  - /config/
[15:54:16] 301 -  305B  - /docs  ->  http://127.0.0.1/docs/
[15:54:16] 200 -  485B  - /docs/
[15:54:17] 200 -  470B  - /dvwa/
[15:54:19] 200 -    1KB - /favicon.ico
[15:54:29] 200 -  699B  - /login.php
```

I.2 - Système de scoring

L'objectif :

La mise en place d'un système de scoring à une place indispensable dans ce projet car il va permettre à un client qui n'est pas un expert en sécurité de visualiser l'état de santé de son serveur web à travers une note globale et un code couleur intuitif.

Comment est faite la notation ?

Notre système repose sur une méthodologie reconnue, le système de notation de l'ANSSI. Plutôt que de simplement compter le nombre de failles, nous évaluons la dangerosité réelle de chaque vulnérabilité découverte en croisant deux critères fondamentaux :

L'Impact (les dégâts si ça arrive) : Mineur, Important, Majeur, Critique.

L'Exploitation (la difficulté pour le pirate) : Très difficile, Difficile, Modérée, Facile.

Tableau abordant le système de notation ANSSI :

Système de notation ANSSI

↓ Impact Exploit. →	Très difficile	Difficile	Modérée	Facile
Mineur	Mineur	Mineur	Important	Majeur
Important	Mineur	Important	Important	Majeur
Majeur	Important	Majeur	Majeur	Critique
Critique	Important	Majeur	Critique	Critique

Le croisement de ces deux données dans la matrice de l'ANSSI nous donne un niveau de gravité précis pour chaque faille trouvée.

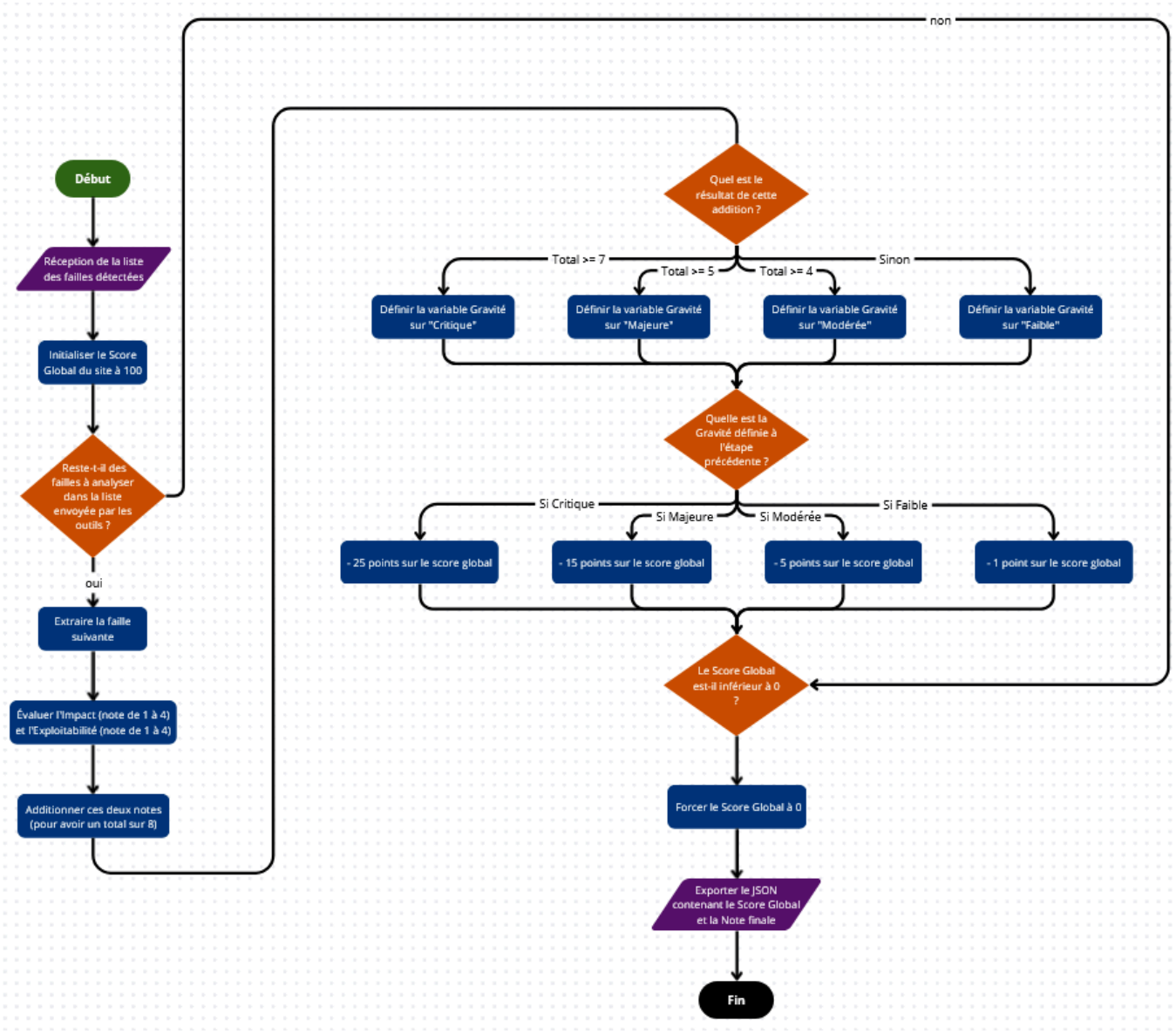
Pour que la note soit parlante pour l'utilisateur, nous avons décidé de partir sur une base de 100 points à laquelle nous allons soustraire un nombre de point en fonction de la gravité de chaque faille :

Faille Critique : - 25 points sur le score global.
Faille Majeure : - 10 points sur le score global.
Faille Modérée : - 5 points sur le score global.
Faille Faible : - 1 point sur le score global.

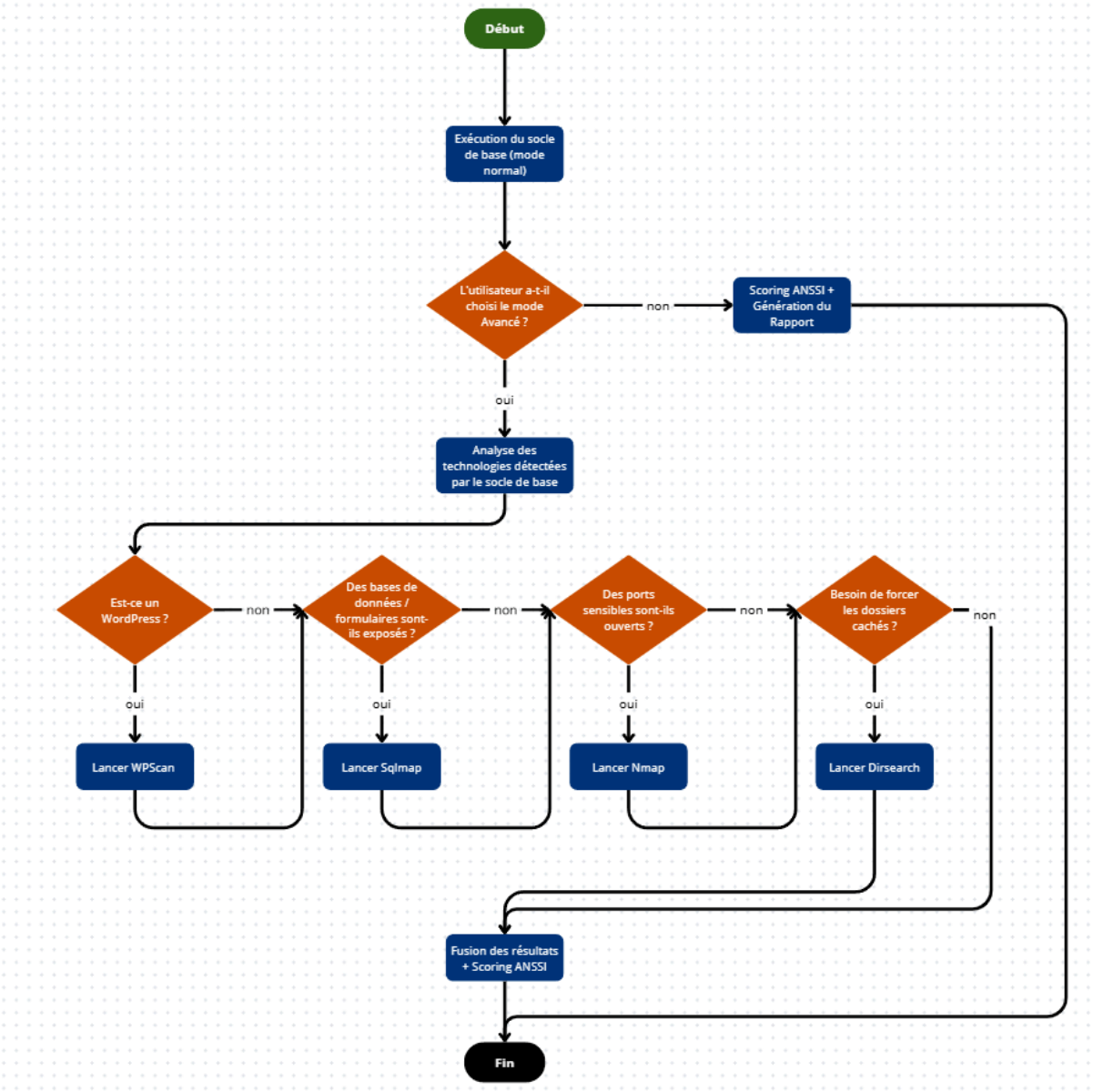
Réalisation de ce système ?

L'algorigramme illustre le processus de calcul automatisé de notre score de sécurité, qui débute avec une note globale initialisée à 100 points. Pour chaque vulnérabilité détectée, le système évalue sa gravité en additionnant un poids d'impact (de 1 à 4) et un poids d'exploitabilité (de 1 à 4), ce qui donne un résultat sur 8. En fonction de ce total, la faille est catégorisée et des points sont déduits du score global : une faille "Critique" (total ≥ 7) retire 25 points , une faille "Majeure" (total ≥ 5) retire 15 points , une faille "Modérée" (total = 4) retire 5 points , et une faille "Faible" en retire 1. Ce cycle se répète de manière itérative pour toutes les failles répertoriées, avec une ultime vérification garantissant que le score global final ne puisse pas descendre en dessous de 0 avant son exportation.

Algorigramme du score ANSSI :



Algorigramme du mode avancé :



Cet algorigramme détaille le fonctionnement de l'orchestrateur. Le processus commence systématiquement par l'exécution du socle de base, c'est-à-dire le mode normal. Si l'utilisateur n'a pas sélectionné l'option avancée, le système s'arrête là, calcule le score ANSSI et génère le rapport final. En revanche, si le mode avancé est activé, l'orchestrateur déclenche des analyses ciblées selon les technologies détectées : WPScan pour WordPress, Sqlmap pour les bases de données, Nmap pour les ports et Dirsearch pour les dossiers cachés. Bien que ce schéma ait été volontairement simplifié pour en faciliter la lecture, le système déploie en réalité un arsenal plus vaste en arrière-plan (incluant Nmap-NSE, Lighthouse, WhatWeb, PyMeta et GitLeaks) pour garantir un audit exhaustif. Enfin, l'ensemble des failles trouvées par ces outils est fusionné afin de calculer le score ANSSI global.

I.3 - Conception de l’architecture de la plateforme SaaS

Avant de commencer la création de la plateforme, il a d’abord fallu identifier les besoins du client, pour ce faire nous avons établie une maquette comportant tous les principaux éléments :

- **Un formulaire de saisie minimaliste**
- **La case à cocher "Analyse mode avancé"**
- **Le Score et le diagramme coloré**
- **La liste vulgarisée des "Principales failles"**
- **Le bouton "Obtenir le rapport PDF"**

Page 1:

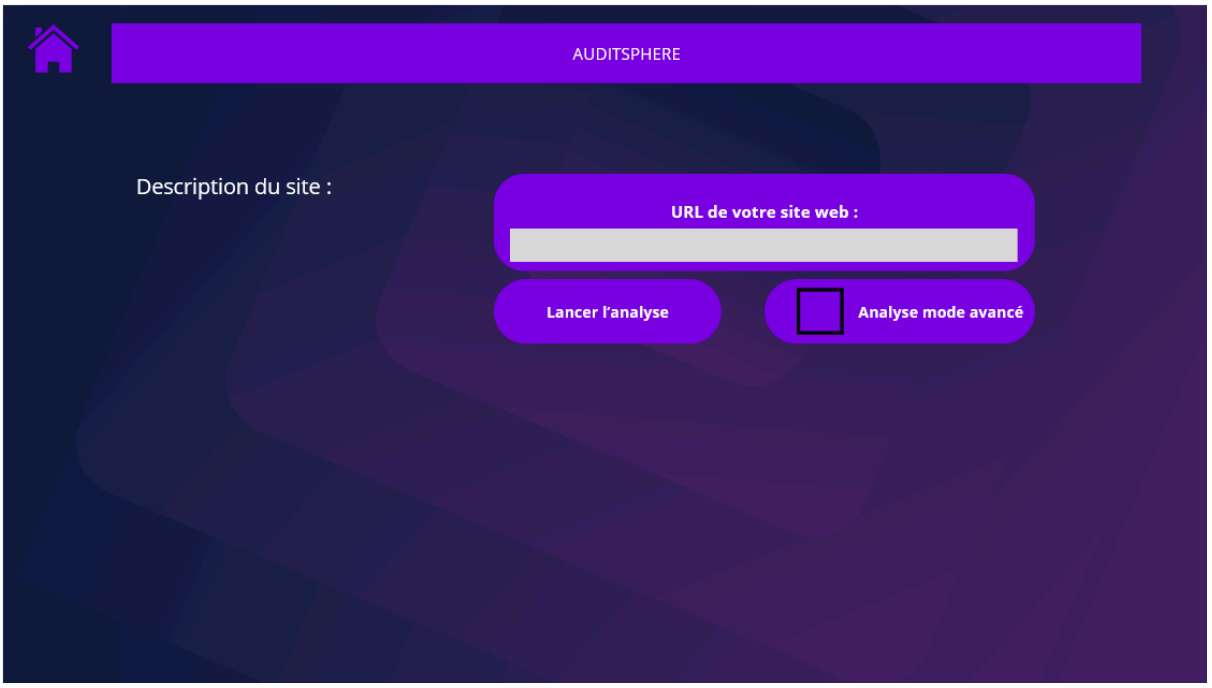
L'idée était de créer une première page qui propose au client, qui n'est pas un expert en sécurité , une interface très épurée pour notre plateforme SaaS.
On a mis en place un formulaire demandant l'URL du site Web pour lancer notre audit en « Black Box » (aucune information sur la cible n'est connue).

L'élément clé de cette page est la case à cocher « Analyse mode avancé ». Elle répond directement à l'exigence d'avoir deux modes de fonctionnement distincts.

Si le client ne coche pas la case, il utilise le mode de base, notre orchestrateur lancera une recherche ciblée sur les vulnérabilités et failles détectables avec les outils simples.

S'il coche la case, on passe en mode avancé pour rechercher l'ensemble des vulnérabilités avec l’utilisation d’outils plus lourds comme LightHouse, Sqlmap, Nmap, Dirsearch.

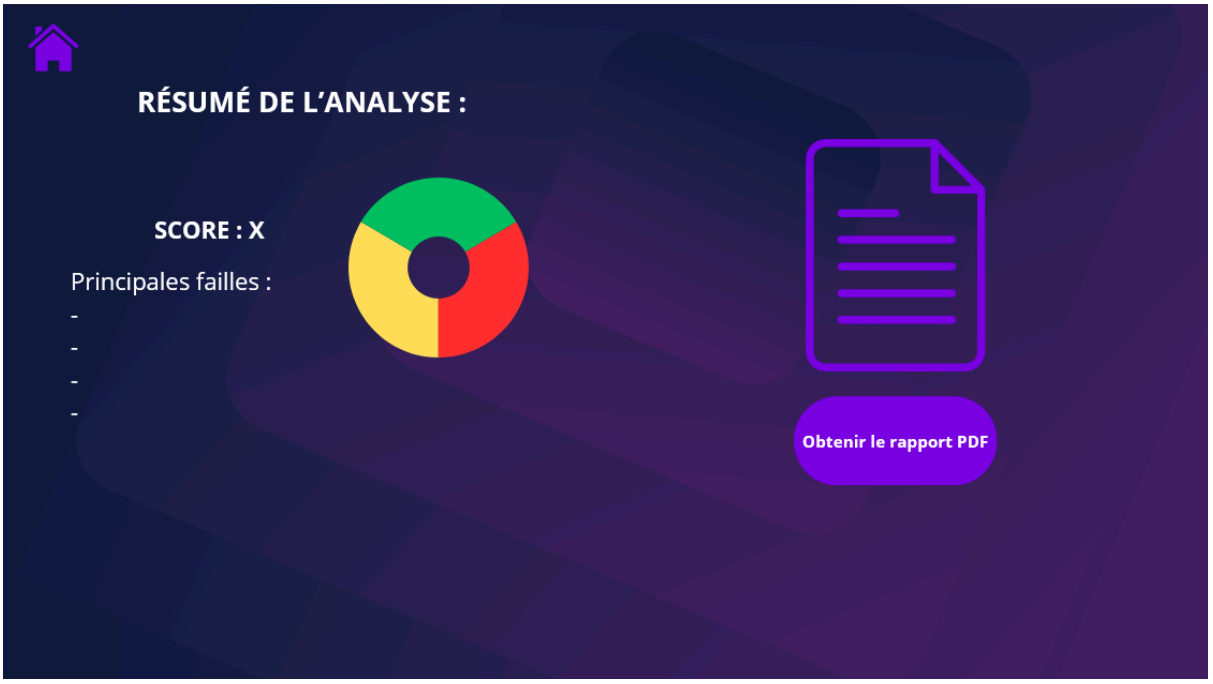
Maquette du site (Première page) :



Page 2:

Sur la seconde page, le but était de concevoir un reporting simple de compréhension pour ne pas noyer le client sous des données techniques, car le rapport technique sera un PDF téléchargeable à part.
Le visuel central avec le SCORE et le diagramme coloré traduit notre système de scoring de risques, basé sur l'estimation de la difficulté d'exploitation et de son impact.
La zone listant les principales failles est là pour assurer la vulgarisation des concepts présentés au sein du rapport.
Enfin, le bouton de téléchargement permet d'obtenir la version PDF pour récupérer le rapport d'audit complet.

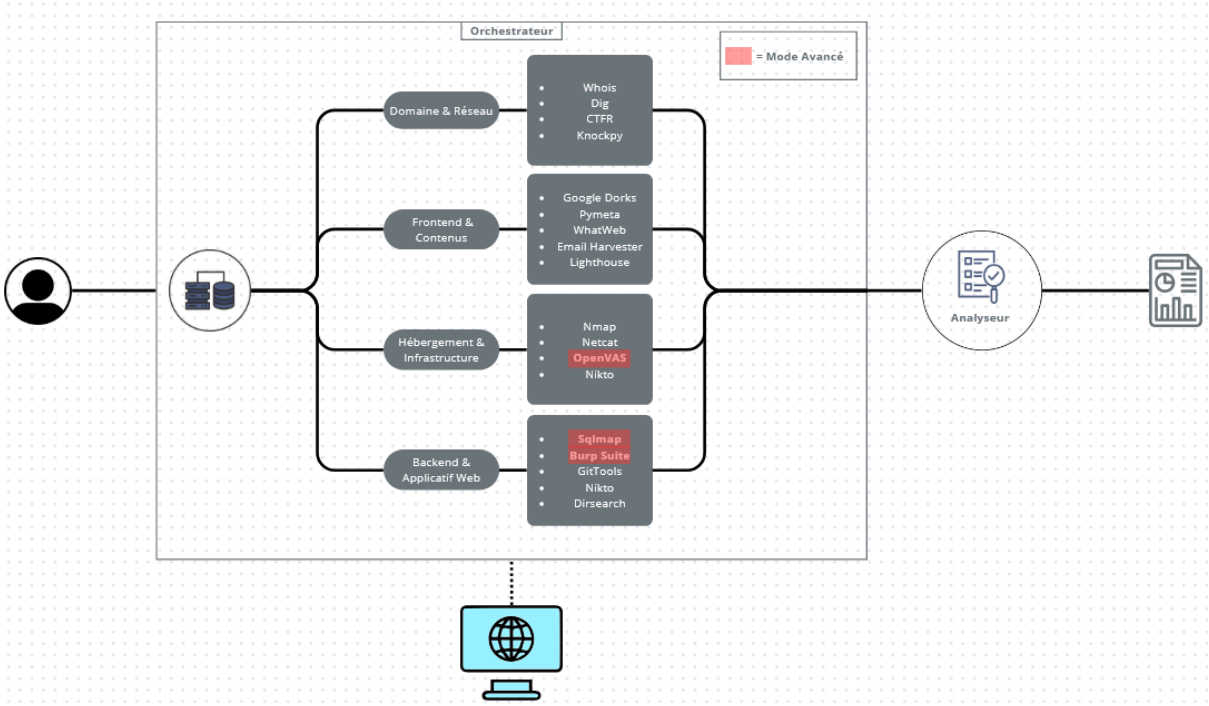
Maquette du site (Seconde page) :



II - Création de l'orchestrateur

L'orchestrateur central est le cerveau automatisé de notre plateforme SaaS, intervenant comme le maillon intermédiaire (middleware) exact entre le formulaire web du client et les scripts de hacking bruts. Son rôle est d'automatiser et de paralléliser tout l'audit en réceptionnant l'URL cible pour distribuer le travail sur quatre pôles d'analyse distincts : le Domaine (ex: Whois), le Frontend (ex: Whatweb), le Backend (ex: SQLmap) et l'Infrastructure (ex: Nmap). C'est lui qui pilote toute la logique de l'attaque : il exécute le scan de base, décide d'activer les outils lourds si le "Mode Avancé" est coché , puis récolte toutes les données brutes générées pour les envoyer à l'analyseur, qui se chargera de calculer le score final .

Schéma logique de l'orchestrateur :



II.1 - Les contraintes technique

La conception de l'orchestrateur a été confrontée à des contraintes techniques majeures, principalement liées au poids réseau et au temps d'exécution des outils . D'un côté, le mode normal se devait d'être rapide et léger : nous y avons donc regroupé des outils furtifs comme Dig, Whois ou WhatWeb, capables de s'exécuter en quelques secondes en n'envoyant qu'une poignée de requêtes. De l'autre côté, le mode avancé posait un véritable défi de performance puisqu'il intègre des scanners lourds comme Nikto (plus de 6 700 requêtes) ou Dirsearch (qui peut générer jusqu'à 100 000 requêtes). La complexité était d'empêcher ce mode de durer des heures, tout en respectant l'obligation stricte d'un audit « White Hat ». Cette contrainte éthique impose en effet une temporisation des requêtes pour ne pas saturer la bande passante du client et causer un déni de service (DDoS) involontaire. Pour compenser cette perte de vitesse obligatoire et garder un mode avancé acceptable, l'orchestrateur a été pensé pour paralléliser les tâches au maximum et conditionner le lancement des outils les plus chronophages uniquement lorsque l'environnement le justifie (comme le déclenchement de SQLmap uniquement si des formulaires sont préalablement détectés).

Afin de définir les bons outils à utiliser pour chaque mode, nous avons dû étudier le poids des outils ainsi que leur vitesse d'exécution afin de réaliser un travail de tri.

Tableau des outils trié PAR POIDS : pré-scan / scan :

Rapidité	Outils Pré-scan	Poids
5) 5 à 15 secondes	Google Dorks	0 requêtes
4) 2 à 5 secondes	CTFR	0 requêtes
8) 15 secondes à 1 minute	Email Harvester	0 requêtes
2) Moins d'une seconde	Whois	1 à 3 requêtes réseau (qqs octets)
1) Quelques millisecondes	Dig	1 à 3 requêtes réseau (qqs octets)
3) 1 à 3 secondes	WhatWeb	1 à 5 requêtes HTTP
9) Quelques minutes	Pymeta	qqs Mo de téléchargement
10) Plusieurs minutes	Knockpy	milliers de requêtes DNS/secondes
	Outils Scan	**Poids**
7) 10 secondes à 1 minute	GitLeaks	1 à 10 Mo
6) 10 à 30 secondes	Lighthouse	50 à 150 requêtes HTTP (1 à 10 Mo de téléchargement)
11) 5 à 15 minutes	Nikto	+ de 6 700 requêtes HTTP spécifiques
12) 15 minutes à plusieurs heures	Dirsearch (Dirb)	10 000 à 100 000+ requêtes HTTP

Classement Rapidité

Léger
Moyen
Lourd

Cette étude nous a permis de classifier les outils selon un code couleur de charge ("Léger", "Moyen", "Lourd") et de les intégrer de manière optimale dans la logique de notre orchestrateur. En séparant strictement les outils de "pré-scan" furtifs (comme Whois ou WhatWeb), qui s'exécutent en moins d'une minute avec un impact réseau minimal, des outils de "scan" profonds (comme Nmap-NSE complet ou SQLMap), dont l'exécution peut prendre plusieurs heures, nous garantissons une flexibilité d'utilisation. Le client a ainsi le choix entre un état des lieux quasi instantané ou un audit en profondeur, tout en assurant que notre plateforme SaaS reste performante, stable, et surtout, respectueuse de l'intégrité de l'infrastructure auditée.

II.2 - La standardisation des données (La fusion des résultats)

L'un des défis techniques majeurs dans la conception de notre orchestrateur réside dans le traitement des résultats générés par notre arsenal d'outils. En effet, comme nous utilisons une multitude de solutions distinctes et Open Source, chacune possède son propre format d'exportation de données :

- GitLeaks et Lighthouse : Produisent nativement des résultats formatés et structurés, souvent en JSON.
- Nmap (et Nmap-NSE) : Génère des résultats bruts en texte ou en XML.
- Secpod : Renvoient de très longues listes en texte brut ou en fichiers .txt classiques (notamment Secpod lors de sa phase de validation des autorisations via les certificats).

Le processus de Parsing (Extraction) Pour que notre algorithme de notation basé sur l'ANSSI puisse fonctionner, il est impossible de lui injecter ces formats disparates. L'orchestrateur intègre donc des scripts d'extraction (le parsing). Dès qu'un outil termine son analyse, l'orchestrateur vient lire le fichier de sortie spécifique, le nettoie, et n'en extrait que les informations cruciales. Par exemple, pour GitLeaks, le script va uniquement récupérer les valeurs "Secret", "RuleID" et le nom du fichier vulnérable.

Script GitLeaks :

```
156        // Process GitLeaks
157        if (ext.gitleaks && ext.gitleaks.status === "success") {
158            let glOut = ext.gitleaks.output;
159            let limitedGlOut = glOut.split('\n').slice(0, 10).join('\n');
160            if (glOut.split('\n').length > 10) limitedGlOut += '\n\n[... Résumé limité à 10 lignes ...]';
161
162            let uiGl = `<details style="margin-top:10px;"><summary style="cursor:pointer; color:#cca8ff; font-weight:bold
163            let pdfGl = `<div style="margin-top:10px; page-break-inside: avoid;"><strong>Secrets détectés :</strong><pre
164            let finalGl = `<div class="hide-in-pdf">${uiGl}</div><div class="show-in-pdf-only" style="display: none;">${p
165
166            if (glOut.includes("leaks found") || glOut.includes("commit")) {
167                discoveredVulns.push({
168                    name: "Fuite de clés secrètes/Tokens (Outil: GitLeaks)",
169                    impact: 4, exploit: 4, // CRITIQUE (-45 pts)
170                    desc: `GitLeaks a détecté des secrets codés en dur (mots de passe, clés API AWS, tokens Stripe) divul
171                });
172            }
173        }
```

Le format unifié final :

Une fois les données extraites de chaque outil, l'orchestrateur procède à la "Fusion des résultats" (comme illustré à la fin de notre algorigramme du mode avancé). Toutes les vulnérabilités trouvées sont traduites et regroupées dans un seul et unique format standardisé (un fichier JSON global propre à notre plateforme).

C'est ce fichier final, propre et uniformisé, qui contient la liste de toutes les failles. Il est ensuite envoyé à notre algorithme de scoring, qui pourra alors lire chaque ligne, attribuer les notes d'Impact et d'Exploitabilité (de 1 à 4), et déduire les points du score global sur 100.

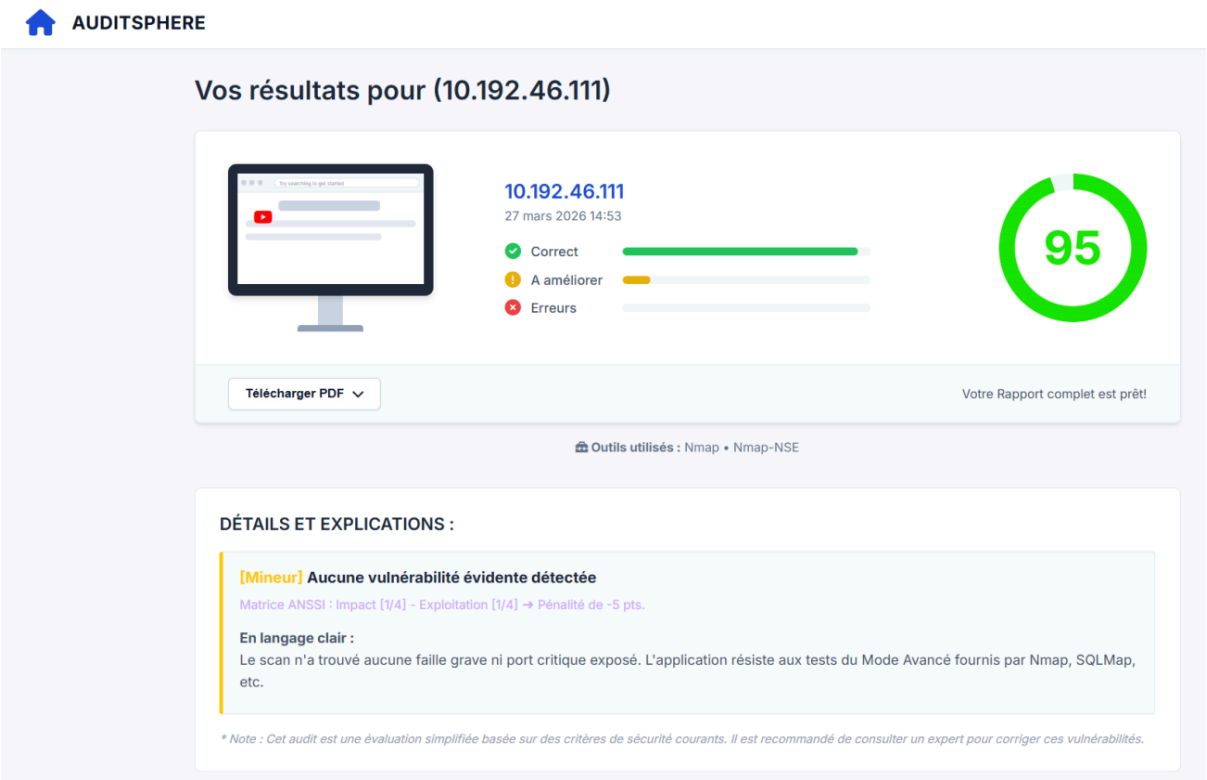
II.3 - L'intégration du Scoring et la génération du livrable PDF

L'ultime étape de notre processus consiste à restituer les données techniques récoltées de manière lisible et actionnable pour notre client, conformément à notre objectif de vulgarisation. Une fois le fichier unifié (JSON) généré et le score ANSSI calculé, la restitution s'effectue en deux temps pour optimiser l'expérience utilisateur :

1. Le tableau de bord interactif (Résumé Web)

Dès la fin de l'analyse, l'orchestrateur n'exporte pas immédiatement un document lourd. Le système affiche d'abord un "Résumé de l'analyse" directement sur la page web de notre plateforme SaaS. Cette interface épurée présente instantanément le score global de sécurité (accompagné de son diagramme coloré) ainsi qu'une liste vulgarisée des principales failles détectées. Le client obtient ainsi un état des lieux immédiat en un coup d'œil.

Résumé de l'analyse depuis la page web :



Le score global : La note sur 100 est affichée en évidence, accompagnée de son code couleur intuitif (allant de "Danger" en rouge à "Parfait" en vert).

Le résumé exécutif : Une liste vulgarisée des failles principales destinée aux chefs de projet ou gérants d'entreprise.

Script du score global :

```
484    // --- ÉTAPE 5 : CALCUL DU SCORE GLOBAL ANSSI ---
485    // Le score part de 100 points maximum pour un audit parfait.
486    let score = 100;
487
488    // On crée un tableau (Array simple) pour se rappeler du nom des failles déjà pénalisées.
489    // Ça évite d'enlever 2 fois 45 points si la même faille web est trouvée 2 fois sur des pages différentes.
490    let nomsDesFaillesDejaVues = [];
491
492    // On crée notre nouveau tableau de travail
493    let faillesGravesCalculees = [];
494
495    // On repasse sur les failles trouvées pour déterminer les points en moins
496    for (let i = 0; i < discoveredVulns.length; i++) {
497        let vuln = discoveredVulns[i];
498
499        // On calcule la catégorie globale (Critique, Majeur, Mineur...) selon le duo d'Impact et d'Exploitation
500        let severity = calculateAnssiSeverity(vuln.impact, vuln.exploit);
501
502        // On récupère le nombre de points à retirer
503        let penalty = getPenaltyPoints(severity);
504
505        // "includes()" vérifie si on a déjà traité ce nom précis :
506        let failleDejaComptee = nomsDesFaillesDejaVues.includes(vuln.name);
507
508        if (failleDejaComptee === true) {
509            penalty = 0; // Si déjà puni, on ne retire plus de points (Pénalité à 0) !
510        } else {
511            // Sinon, c'est nouveau. On l'enregistre dans nos archives pour la prochaine fois
512            nomsDesFaillesDejaVues.push(vuln.name);
513            // On met à jour le score global
514            score = score - penalty;
515        }
```

2. La génération du livrable PDF à la demande

C'est uniquement à la demande de l'utilisateur, par une action sur le bouton « Obtenir le rapport PDF », que l'orchestrateur déclenche le processus de création du document complet. Un script lit alors notre fichier JSON final et vient injecter dynamiquement les variables (score, liste des vulnérabilités, détails techniques) dans un modèle de document prédéfini (Template). Une librairie logicielle de conversion (comme **html2pdf.js**) transforme ensuite cette vue dynamique en un fichier PDF statique, léger et professionnel.

Appel de la librairie pour la conversion PDF :

```
// 5. Génération du PDF avec html2pdf.js
bindPdfGeneration(urlCible, donutChart);
```

3. La structure du rapport complet

Le document PDF téléchargé est conçu pour être transmis et compris par les différentes équipes du client :

- Pour les décideurs : La première partie reprend le résumé visuel de l'interface web (score global et explications simples pour les gérants ou chefs de projet).

Première partie du rapport PDF :

- Pour les techniciens (Annexe technique) : La suite du rapport détaille précisément les vulnérabilités pour permettre leur remédiation technique. Par exemple, si GitLeaks a détecté une fuite de mot de passe, le rapport indiquera précisément le nom du fichier vulnérable et la ligne exacte à corriger.

Annexe technique réservé aux détails des vulnérabilités :

DÉTAILS ET EXPLICATIONS :

[Important] Port ouvert détecté : 8080 (http-proxy)

Matrice ANSSI : Impact [2/4] - Exploitation [2/4] → Pénalité de -15 pts.

En langage clair :
Le port 8080 est ouvert pour recevoir des connexions externes via Service Inconnu.
(Service standard autorisé sur ce port, pas de pénalité intrinsèque)

Logs d'audit combinés pour ce port :

Détails techniques (Script: http-methods) :

```
Supported Methods: GET HEAD POST OPTIONS
```

Détails techniques (Script: http-headers) :

```
Access-Control-Allow-Origin: *
  X-Content-Type-Options: nosniff
  X-Frame-Options: SAMEORIGIN
  Feature-Policy: payment 'self'
  X-Recruiting: /#/jobs
  Accept-Ranges: bytes
  Cache-Control: public, max-age=0
  Last-Modified: Sun, 29 Mar 2026 14:03:31 GMT
  ETag: W/"124fa-19d39e832de"
  Content-Type: text/html; charset=UTF-8
  Content-Length: 75002
  Vary: Accept-Encoding
  Date: Sun, 29 Mar 2026 14:30:26 GMT
  Connection: close

  (Request type: HEAD)
```

Détails techniques (Script: fingerprint-strings) :

```
FourOhFourRequest:
    HTTP/1.1 200 OK
    Access-Control-Allow-Origin: *
```

C'est la mise à disposition de ce fichier PDF téléchargeable qui clôture définitivement le cycle de notre audit automatisé.

Conclusion

La conception de l'orchestrateur AuditSpher répond avec succès au défi technique et stratégique de ce projet : démocratiser l'audit de cybersécurité. En automatisant le déploiement conditionnel d'outils Open Source reconnus (tels que Nmap, WPScan ou GitLeaks), nous avons réussi à créer un moteur de scan complet, capable d'analyser en profondeur les quatre piliers de la surface d'attaque d'une cible, tout en contournant les freins juridiques et financiers liés aux licences propriétaires.

Le véritable accomplissement de ce travail réside toutefois dans le traitement de l'information. La capacité de notre système à extraire des données brutes hétérogènes, à les standardiser, puis à les évaluer via un algorithme de scoring inspiré de l'ANSSI, permet de combler le fossé entre la technique pure et le client final. Avec son interface web intuitive et sa génération de rapports PDF vulgarisés sur demande, AuditSpher atteint son objectif principal : rendre la cybersécurité parfaitement compréhensible et actionnable pour les décideurs, même non-experts.

À l'avenir, cette architecture modulaire laisse la porte ouverte à de nombreuses perspectives d'évolution. L'intégration de nouveaux outils de scan spécifiques ou l'optimisation continue de la parallélisation des tâches pourraient encore accroître la rapidité et la pertinence des audits. En définitive, AuditSpher prouve qu'il est possible d'offrir le compromis parfait entre l'accessibilité de l'Open Source et l'exigence d'une solution d'audit professionnelle.